Experiment 7: Implement JWT Tokens for Authentication

Aim

To implement JSON Web Token (JWT) based authentication for secure API access and session management without server-side session storage.

Software Requirements

- Python 3.10+
- Flask
- Flask-JWT-Extended
- pymongo
- bcrypt
- python-dotenv

Theory

JWT (JSON Web Token) is a stateless authentication mechanism where tokens are self-contained and signed. Unlike session-based authentication that stores user data on the server, JWT tokens encode user information that can be verified using a secret key. A JWT consists of three parts: Header (algorithm info), Payload (user claims), and Signature (verification). When a user logs in, the server generates a JWT, and the client sends it with each request in the Authorization header. The server verifies the token's signature without needing to query a database for session data, making it ideal for APIs and scalable applications.

Procedure

Step 1: Create Project Directory

```
mkdir exp7_jwt_auth cd
exp7_jwt_auth
```

Step 2: Create and Activate Virtual Environment

```
python -m venv venv
```

Windows:

```
venv\Scripts\activate
```

Mac/Linux:

```
source venv/bin/activate
```

Step 3: Install Required Packages

```
pip install flask flask-jwt-extended pymongo bcrypt python-dotenv
```

Step 4: Create .env File

Create in root directory:

```
MONGO_URI=mongodb+srv://admin:YOUR_PASSWORD@cluster0.xxxxx.mongodb.net/
JWT_SECRET_KEY=your_very_long_random_secret_key_here_minimum_32_characters_recommended
```

Step 5: Create Project Structure

```
mkdir templates mkdir
static mkdir static/css
mkdir static/js
```

Step 6: Create app.py

```
from flask import Flask, request, jsonify, render_template from
flask_jwt_extended import (
    JWTManager, create_access_token, create_refresh_token,
    jwt_required, get_jwt_identity, get_jwt
)
from pymongo import
MongoClient import bcrypt
import os
from dotenv import load_dotenv
from datetime import timedelta

load_dotenv()

app = Flask(__name__

) # JWT

Configuration
app.config['JWT_SECRET_KEY'] = os.getenv('JWT_SECRET_KEY')
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(hours=1)
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(days=30)

jwt = JWTManager(app)

# MongoDB connection
client =
MongoClient(os.getenv('MONGO_URI')) db =
client['fullstack_ai_lab'] users_collection =
db['users']
revoked_tokens_collection = db['revoked_tokens']
```

```
@app.route('/')
def home():
    return render_template('index.html')

@app.route('/api/signup', methods=['POST'])
def signup():
    """Register a new user"""
    try:
        data = request.get_json() email
        = data.get('email') password =
        data.get('password')

        # Validation
        if not email or not password:
            return jsonify({'error': 'Email and password required'}), 400

        if len(password) < 6:
            return jsonify({'error': 'Password must be at least 6 characters'}), 400

        # Check if user exists
        if  users_collection.find_one({'email':  email}):
            return jsonify({'error': 'Email already registered'}), 400

        # Hash password
        hashed_password  =  bcrypt.hashpw(password.encode('utf-8'),  bcrypt.gensalt())

# Create user users_collection.insert_one({
            'email': email,
            'password': hashed_password,
            'created_at': datetime.utcnow()
        })

        return jsonify({'message': 'User registered successfully'}), 201

    except Exception as e:
        return jsonify({'error': str(e)}), 500

@app.route('/api/login', methods=['POST'])
def login():
    """Login and receive JWT
    tokens""" try:
        data = request.get_json()
```

```
        email = data.get('email') password
        = data.get('password')

        if not email or not password:
            return jsonify({'error': 'Email and password required'}), 400

        # Find user
        user = users_collection.find_one({'email': email})

        if not user or not bcrypt.checkpw(password.encode('utf-8'), user['password']):
            return jsonify({'error': 'Invalid credentials'}), 401

        # Create tokens
        access_token = create_access_token(identity=email)
        refresh_token = create_refresh_token(identity=email)

        return jsonify({
            'access_token': access_token,
            'refresh_token': refresh_token,
            'user': email
        }), 200

    except Exception as e:
        return jsonify({'error': str(e)}), 500

@app.route('/api/refresh', methods=['POST'])
@jwt_required(refresh=True)
def refresh():
    """Refresh access token using refresh token"""
    current_user = get_jwt_identity()
    access_token = create_access_token(identity=current_user)
    return jsonify({'access_token': access_token}), 200

@app.route('/api/protected', methods=['GET'])
@jwt_required()
```

```
def protected():
    """Protected route requiring valid JWT"""
    current_user = get_jwt_identity()
    return jsonify({
        'message': 'Access granted to protected resource',
        'user': current_user,
        'timestamp': datetime.utcnow().isoformat()
    }), 200

@app.route('/api/logout', methods=['POST'])
@jwt_required()
def logout():
    """Logout by revoking the JWT token"""
    jti = get_jwt()['jti']  # JWT ID
    revoked_tokens_collection.insert_one({'jti': jti})
    return jsonify({'message': 'Successfully logged out'}), 200

# Check if token is revoked
@jwt.token_in_blocklist_loader
def check_if_token_revoked(jwt_header, jwt_payload):
    jti = jwt_payload['jti']
    token = revoked_tokens_collection.find_one({'jti': jti}) return
    token is not None

# JWT error handlers
@jwt.expired_token_loader
def expired_token_callback(jwt_header, jwt_payload):
    return jsonify({'error': 'Token has expired'}), 401

@jwt.invalid_token_loader
def invalid_token_callback(error):
    return jsonify({'error': 'Invalid token'}), 401

@jwt.unauthorized_loader
def missing_token_callback(error):
    return jsonify({'error': 'Authorization header required'}), 401 from

datetime import datetime

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

Step 7: Create templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JWT Authentication Demo</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/jwt.css') }}">
</head>
<body>
```

```
    <div class="container">
        <h1>□ JWT Authentication System</h1>
        <p class="subtitle">Stateless token-based authentication with Flask</p>

        <div id="authSection">
            <div class="tab-buttons">
                <button class="tab-btn active" onclick="showTab('login')">Login</button>
                <button class="tab-btn" onclick="showTab('signup')">Sign Up</button>
            </div>

            <div id="loginTab" class="tab-content active">
                <h2>Login</h2>
                <form id="loginForm">
                    <div class="form-group">
                        <label for="loginEmail">Email</label>
                        <input type="email" id="loginEmail" placeholder="your@email.com"

required>
                    </div>
                    <div class="form-group">
                        <label for="loginPassword">Password</label>
                        <input type="password" id="loginPassword" placeholder="Enter
password" required>
                    </div>
                    <button type="submit" class="btn">Login</button>
                </form>
            </div>
```

```
required>        <div id="signupTab" class="tab-content">
                     <h2>Sign Up</h2>
                     <form id="signupForm">
                         <div class="form-group">
                             <label for="signupEmail">Email</label>
                             <input type="email" id="signupEmail" placeholder="your@email.com"

                         </div>
                         <div class="form-group">
                             <label for="signupPassword">Password</label>
                             <input type="password" id="signupPassword" placeholder="Min 6
```

```
characters" required>
                    </div>
                    <div class="form-group">
                        <label for="signupConfirm">Confirm Password</label>
                        <input type="password" id="signupConfirm" placeholder="Confirm
password" required>
                    </div>
                    <button type="submit" class="btn">Sign Up</button>
                </form>
            </div>
        </div>

        <div id="dashboardSection" style="display:none;">
            <h2>Welcome, <span id="userEmail"></span>!</h2>
```

```
                <div class="dashboard-grid">
                    <div class="card">
                        <h3>Access Token</h3>
                        <div class="token-display" id="accessTokenDisplay"></div>
                        <small>Valid for 1 hour</small>
                    </div>

                    <div class="card">
                        <h3>Refresh Token</h3>
                        <div class="token-display" id="refreshTokenDisplay"></div>
                        <small>Valid for 30 days</small>
                    </div>
                </div>

                <div class="actions">
                    <button class="btn-action" onclick="testProtectedRoute()">Test Protected
Route</button>
                    <button class="btn-action" onclick="refreshToken()">Refresh Access
Token</button>
                    <button class="btn-action btn-logout" onclick="logout()">Logout</button>
                </div>

                <div id="responseArea" style="display:none;"></div>
            </div>

            <div id="message" class="message"></div>
        </div>

        <script src="{{ url_for('static', filename='js/jwt.js') }}"></script>
</body>
</html>
```

Step 8: Create static/css/jwt.css

```
* {
    margin: 0;
    padding: 0;
    box-sizing:  border-box;
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; background: linear-
    gradient(135deg, #667eea 0%, #764ba2 100%); min-height:  100vh;
    display: flex;
    justify-content: center; align-items:
    center; padding: 20px;
}

.container {
    background: white; padding:
    40px;
```

```
    border-radius:  10px;
    box-shadow: 0 10px 30px rgba(0,0,0,0.3);
    max-width: 700px;
    width: 100%;
}

h1 {
    text-align: center;
    color: #333;
    margin-bottom:
    5px;
}

.subtitle {
    text-align: center; color:
    #666;
    margin-bottom:
    30px; font-size:
    14px;
}

.tab-buttons {
    display: flex; gap:
    10px;
```

```
        margin-bottom:  25px;
  }

  .tab-btn {
        flex: 1;
        padding: 12px;
        border: 2px solid #ddd;
        background: white;
        color: #333;
        cursor: pointer;
        border-radius: 5px;
        font-size: 16px;
        font-weight: 600;
        transition: all 0.3s;
  }

  .tab-btn.active {
        background: #667eea;
        color: white;
        border-color:  #667eea;
  }

  .tab-btn:hover  {
        border-color:  #667eea;
  }

  .tab-content {
        display: none;
        animation: fadeIn 0.3s;
  }

.tab-content.active { display: block;
  }

  @keyframes fadeIn {
        from { opacity: 0; }
        to { opacity: 1; }
  }

  h2 {
        color: #333;
        margin-bottom:
        20px; font-size:
        22px;
```

```
}

.form-group {
    margin-bottom:  15px;
}

label {
    display: block;
    margin-bottom:
    5px; color: #555;
    font-weight: 600;
    font-size: 14px;
}

input {
    width: 100%;
    padding: 12px;
    border: 2px solid #ddd;
    border-radius: 5px;
    font-size: 14px;
    transition: border-color 0.3s;
}

input:focus {
    outline: none;
    border-color:  #667eea;
}

.btn {
    width: 100%;
    padding: 12px;
    background:
    #667eea; color:
    white; border: none;
    border-radius: 5px;
    font-size: 16px;
    font-weight: 600;
    cursor: pointer;
    transition: background 0.3s;
}
```

```
.btn:hover {
    background: #764ba2;
}

.dashboard-grid {
    display: grid;
    grid-template-columns: 1fr 1fr; gap:
    20px;
    margin-bottom:  30px;
}

.card {
    background: #f8f9ff;
    padding: 20px;
    border-radius: 8px;
    border: 2px solid #e0e0ff;
}

.card h3 {
    color: #667eea;
    margin-bottom:
    10px; font-size:
    16px;
}

.token-display {
    background:
    #1a1a1a; color:
    #0f0; padding:
    12px; border-radius:
    5px;
    font-family: 'Courier New', monospace;
    font-size: 11px;
    word-break: break-all;
    max-height: 120px;
    overflow-y: auto;
    margin: 10px 0;
}

.card small {
    color: #999;
```

```
    font-size: 12px;
}

.actions {
    display: grid;
    grid-template-columns: repeat(3, 1fr); gap:
    10px;
    margin-bottom:  20px;
}

.btn-action {
    padding: 10px;
    background: #28a745;
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    font-size: 14px;
    font-weight: 600;
    transition: background 0.3s;
}

.btn-action:hover {
    background: #218838;
}

.btn-action.btn-logout {
    background:
    #dc3545;
}

.btn-action.btn-logout:hover {
    background: #c82333;
}

#responseArea {
    background: #e7f3ff;
    border: 2px solid #b3d9ff;
    border-radius: 5px;
    padding: 15px;
    margin-top: 20px;
}
```

```
#responseArea h3 {
    color: #0056b3;
    margin-bottom:
    10px;
}
#responseArea pre {
    background:
    #1a1a1a; color:
    #0f0; padding:
    10px; border-radius:
    3px; overflow-x:
    auto; font-size:
    12px;
}
.message {
    margin-top: 20px;
    padding: 15px;
    border-radius: 5px;
    display: none;
    animation: slideIn 0.3s;
}
@keyframes slideIn {
    from { transform: translateY(-10px); opacity: 0; }
```

```
        to { transform: translateY(0); opacity: 1; }
}

.message.success { background:
        #d4edda; color: #155724;
        border: 2px solid #c3e6cb;
}

.message.error { background:
        #f8d7da; color: #721c24;
        border: 2px solid #f5c6cb;
}

@media (max-width: 600px) {
        .dashboard-grid {
                grid-template-columns: 1fr;
        }

        .actions {
                grid-template-columns: 1fr;
        }
}
```

Step 9: Create static/js/jwt.js

```
let accessToken = null;
let refreshTokenValue = null;
const BASE_URL = 'http://localhost:5000';

function showTab(tab) {
        document.querySelectorAll('.tab-content').forEach(t => t.classList.remove('active')); document.querySelectorAll('.tab-
        btn').forEach(b => b.classList.remove('active'));

        if (tab === 'login') { document.getElementById('loginTab').classList.add('active');
                document.querySelectorAll('.tab-btn')[0].classList.add('active');
        } else {
                document.getElementById('signupTab').classList.add('active'); document.querySelectorAll('.tab-
                btn')[1].classList.add('active');
        }
}

function showMessage(text, type) {
        const messageDiv = document.getElementById('message'); messageDiv.textContent = text;
        messageDiv.className = `message ${type}`;
        messageDiv.style.display = 'block';

        setTimeout(() => { messageDiv.style.display = 'none';
        }, 5000);
```

```
}

document.getElementById('signupForm').addEventListener('submit', async (e) => {
    e.preventDefault();

    const email = document.getElementById('signupEmail').value;
    const password =
    document.getElementById('signupPassword').value;
    const confirmPassword = document.getElementById('signupConfirm').value;

    if (password !== confirmPassword) {
        showMessage('Passwords do not match', 'error');
        return;
    }

    if (password.length < 6) {
        showMessage('Password must be at least 6 characters', 'error');
        return;
    }

    try {
        const response = await fetch(`${BASE_URL}/api/signup`, {
            method: 'POST',
            headers: {'Content-Type': 'application/json'},
            body: JSON.stringify({email, password})
        });

        const data = await response.json(); if

        (response.ok) {
            showMessage('Registration successful! Please login.', 'success');
            document.getElementById('signupForm').reset();
            showTab('login');
        } else {
            showMessage(data.error || 'Signup failed', 'error');
        }
    } catch (error) {
        showMessage('Error: ' + error.message, 'error');
    }
});

document.getElementById('loginForm').addEventListener('submit', async (e) => {
    e.preventDefault();
```

```
    const email = document.getElementById('loginEmail').value;
    const password =
    document.getElementById('loginPassword').value;

    try {
        const response = await fetch(`${BASE_URL}/api/login`, {
            method: 'POST',
            headers: {'Content-Type': 'application/json'},
            body: JSON.stringify({email, password})
        });



        const data = await response.json();

        if (response.ok) {
            accessToken = data.access_token;
            refreshTokenValue = data.refresh_token;

            document.getElementById('authSection').style.display = 'none';
            document.getElementById('dashboardSection').style.display = 'block';
            document.getElementById('userEmail').textContent = data.user;
            document.getElementById('accessTokenDisplay').textContent =
accessToken.substring(0, 50) + '...';
            document.getElementById('refreshTokenDisplay').textContent =
refreshTokenValue.substring(0, 50) + '...';

            showMessage('Login successful!', 'success');
        } else {
            showMessage(data.error || 'Login failed', 'error');
        }
    } catch (error) {
        showMessage('Error: ' + error.message, 'error');
    }
});

async function testProtectedRoute() {
    try {
        const response = await fetch(`${BASE_URL}/api/protected`, {
            method: 'GET',
            headers: {
                'Authorization': `Bearer ${accessToken}`
```

```
            }
        });

        const data = await response.json();

        const responseArea = document.getElementById('responseArea');
        responseArea.style.display = 'block';
        responseArea.innerHTML = `
            <h3>Protected Route Response:</h3>
            <pre>${JSON.stringify(data, null, 2)}</pre>
        `;

        if (response.ok) {
            showMessage('Access granted!', 'success');
        } else {
            showMessage('Access denied: ' + data.error, 'error');
        }
    } catch (error) {
        showMessage('Error: ' + error.message, 'error');
    }
}

async function refreshToken() {
    try {
        const response = await fetch(`${BASE_URL}/api/refresh`, {
            method: 'POST',
            headers: {
                'Authorization': `Bearer ${refreshTokenValue}`
            }
        });

        const data = await response.json(); if

        (response.ok) {
            accessToken = data.access_token;
            document.getElementById('accessTokenDisplay').textContent =
accessToken.substring(0, 50) + '...';
            showMessage('Token refreshed successfully!', 'success');
        } else {
            showMessage('Failed to refresh token', 'error');
        }
    } catch (error) {
        showMessage('Error: ' + error.message, 'error');
    }
}
```

```
async function logout() { try
    {
        const response = await fetch(`${BASE_URL}/api/logout`, {
            method: 'POST',
            headers: {
                'Authorization': `Bearer ${accessToken}`
            }
        });

        if (response.ok) {
            accessToken = null;
            refreshTokenValue = null;

            document.getElementById('authSection').style.display = 'block';
            document.getElementById('dashboardSection').style.display = 'none';
            document.getElementById('loginForm').reset();
            document.getElementById('responseArea').style.display = 'none';

            showMessage('Logged out successfully!', 'success');
        }
    } catch (error) {
        showMessage('Error: ' + error.message, 'error');
    }
}
```

Step 10: Create requirements.txt

```
pip freeze > requirements.txt
```

Step 11: Run the Application

```
python app.py
```

Step 12: Test JWT Authentication

1. Open browser at `http://localhost:5000`
2. Click Sign Up tab
3. Register with email and password (min 6 chars)
4. Switch to Login tab
5. Login with your credentials
6. View the Access Token and Refresh Token
7. Click "Test Protected Route" to verify token works
8. Click "Refresh Access Token" to get a new token

9. Click "Logout" to revoke the token
10. Try accessing protected route again (should fail with "Authorization header required")

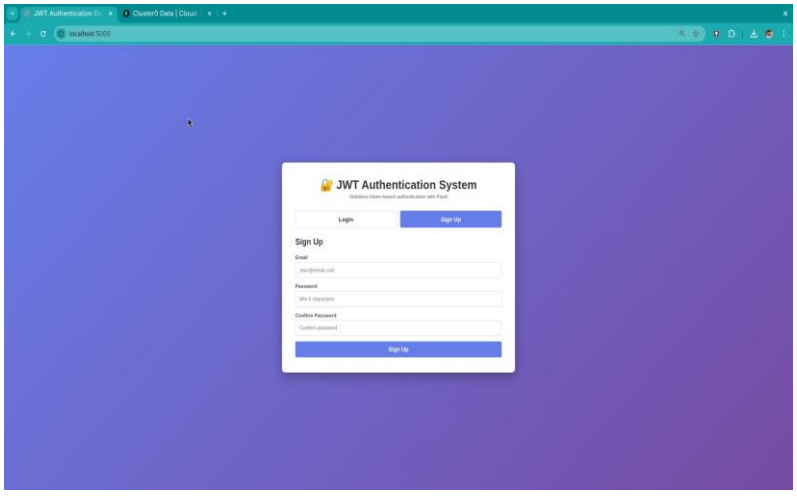
Output

A complete JWT authentication system with:

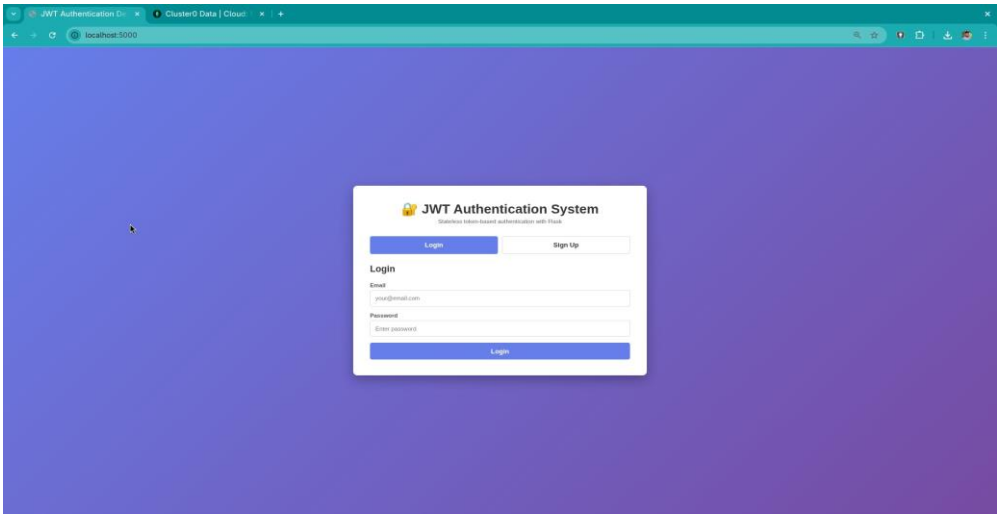
- User registration with password validation
- Login with JWT token generation
- Access and Refresh token mechanism
- Protected routes requiring valid JWT
- Token refresh functionality
- Token revocation on logout
- Token blocklist stored in MongoDB
- Stateless authentication
- Clear token display for debugging
- Comprehensive error messages

Screen-shots

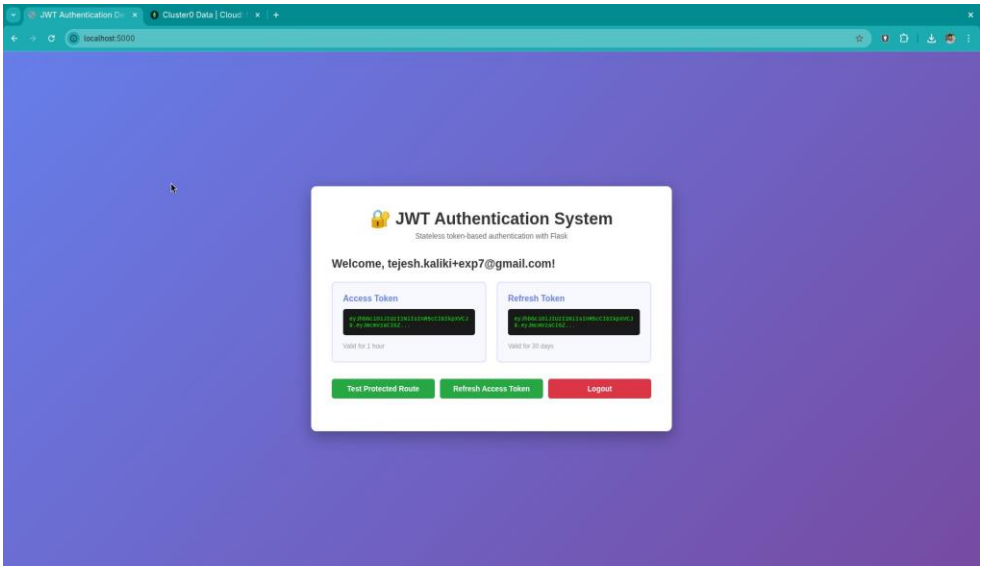
Sign-up page



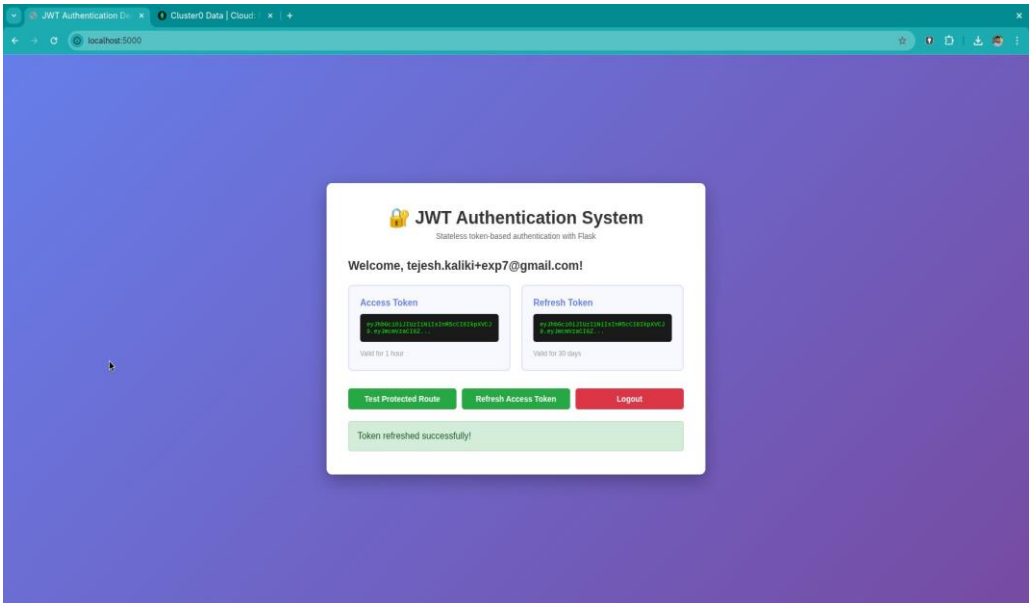
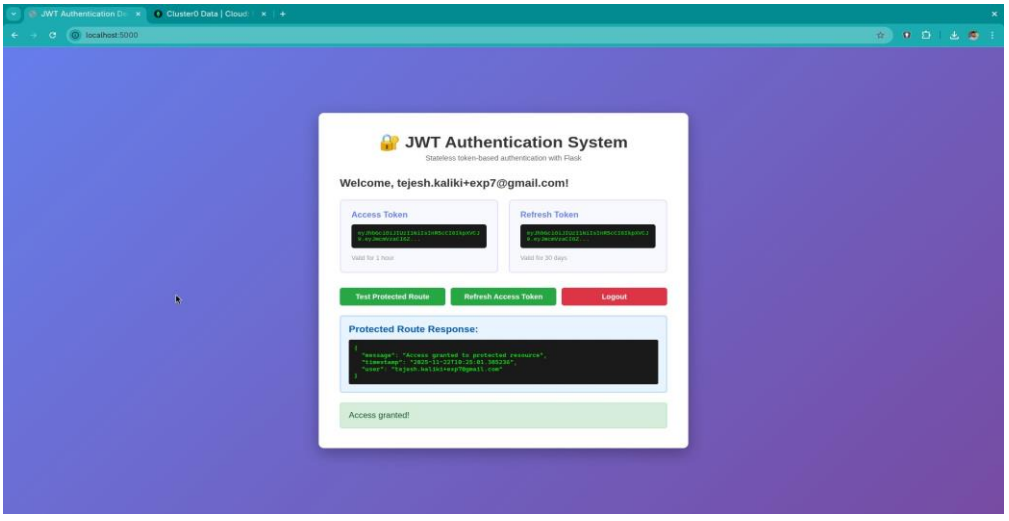
Login Page:



After successful login:



After clicking "Refresh Access Token":



After clicking "Test protected route":